

Testing

Three tools do most of the work in a Java project. JUnit runs the tests, Mockito replaces the parts we are not testing, and MockMvc drives a controller without starting a server.

A good order to learn them in:

The shape of a test → JUnit 5 → Assertions → Parameterized tests
→ Mockito → Verifying → MockMvc → What to mock

1. The shape of a test

Every test has three parts, whatever they are called.

```
@Test
void withdrawingMoreThanTheBalanceIsRejected() {
    // given
    Account account = new Account(new BigDecimal("100"));

    // when, then
    assertThatThrownBy(() -> account.withdraw(new BigDecimal("500")))
        .isInstanceOf(InsufficientFundsException.class);
}
```

Part	Also called	What it does
Arrange	Given	Builds the situation
Act	When	Runs the one thing being tested
Assert	Then	Checks the result

The name should say what is being proved. A failing build that reports `withdrawingMoreThanTheBalanceIsRejected` has already explained itself. `testWithdraw2` has not.

One behavior per test. A test that checks four unrelated things reports one failure and hides three.

2. JUnit 5

JUnit 5 is three parts: the Platform that runs tests, Jupiter that provides the annotations, and Vintage that runs old JUnit 4 tests. Everything below is Jupiter.

Annotation	When it runs
<code>@Test</code>	The test itself

Annotation	When it runs
@BeforeEach, @AfterEach	Around every test
@BeforeAll, @AfterAll	Once for the class, and must be <code>static</code>
@DisplayName	Names the test in the report
@Nested	Groups related tests in an inner class
@Disabled	Skips it, and should carry a reason
@Tag	Selects groups of tests at run time

```

class LifecycleTest {

    @BeforeAll
    static void beforeAll() { System.out.println("beforeAll"); }

    @BeforeEach
    void beforeEach() { System.out.println("  beforeEach"); }

    @Test void first() { System.out.println("    first"); }
    @Test void second() { System.out.println("    second"); }

    @AfterEach
    void afterEach() { System.out.println("  afterEach"); }

    @AfterAll
    static void afterAll() { System.out.println("afterAll"); }
}

```

Output:

```

beforeAll
  beforeEach
    first
  afterEach
  beforeEach
    second
  afterEach
afterAll

```

`@BeforeEach` runs again for the second test, which is the point: each test starts from a clean state. JUnit does not promise the order of the two tests, so a test that depends on another having run first is already broken.

`@Nested` groups cases that share a setup:

```

class AccountTest {

    @Nested
    @DisplayName("when the balance is 100")
    class WithBalance {

        Account account = new Account(new BigDecimal("100"));

        @Test void allowsASmallerWithdrawal() { }
        @Test void rejectsALargerWithdrawal() { }
    }
}

```

3. Assertions

JUnit supplies assertions, and most projects use AssertJ instead because it reads in one direction and its failure messages say more.

```

// JUnit
assertEquals("Ravi", employee.getName());
assertTrue(employees.isEmpty());
assertThrows(InsufficientFundsException.class, () -> account.withdraw(500));

// AssertJ
assertThat(employee.getName()).isEqualTo("Ravi");
assertThat(employees).isEmpty();
assertThatThrownBy(() -> account.withdraw(500))
    .isInstanceOf(InsufficientFundsException.class)
    .hasMessageContaining("balance");

```

AssertJ chains, which removes most loops from tests:

```

assertThat(employees)
    .hasSize(3)
    .extracting(Employee::getName)
    .containsExactly("Ravi", "Amit", "Priya");

```

assertEquals takes the expected value first. Reversing the arguments makes every failure message say the opposite of what happened, and nothing catches it. AssertJ has no such order to remember.

assertAll reports every failure instead of stopping at the first:

```
assertAll(
    () -> assertThat(response.name()).isEqualTo("Ravi"),
    () -> assertThat(response.email()).isEqualTo("ravi@example.com"),
    () -> assertThat(response.salary()).isEqualToByComparingTo("75000"));
```

4. Parameterized tests

Five near-identical tests should usually be one test with five cases.

```
@ParameterizedTest
@ValueSource(strings = {"", " ", "\t"})
void blankNamesAreRejected(String name) {
    assertThatThrownBy(() -> new Employee(name))
        .isInstanceOf(IllegalArgumentException.class);
}
```

```
@ParameterizedTest
@CsvSource({
    "100, 50, 50",
    "100, 100, 0",
    "100, 0, 100"
})
void withdrawLeavesTheRightBalance(int start, int amount, int expected) {
    Account account = new Account(start);

    account.withdraw(amount);

    assertThat(account.balance()).isEqualTo(expected);
}
```

Source	Supplies
<code>@ValueSource</code>	One simple value per run
<code>@CsvSource</code>	Several values per run, written inline
<code>@CsvFileSource</code>	The same, from a file
<code>@EnumSource</code>	Every constant of an enum
<code>@MethodSource</code>	Anything, from a static method returning a <code>Stream</code>
<code>@NullAndEmptySource</code>	<code>null</code> and the empty value

`@MethodSource` is the one for objects:

```

@ParameterizedTest
@MethodSource("invalidRequests")
void invalidRequestsAreRejected(EmployeeRequest request) { }

static Stream<EmployeeRequest> invalidRequests() {
    return Stream.of(
        new EmployeeRequest(null, "ravi@example.com", TEN),
        new EmployeeRequest("Ravi", "not-an-email", TEN));
}

```

Each case is reported separately, so a failure names the value that broke it.

5. Mockito

A unit test should exercise one class. Mockito supplies stand-ins for the others, so a service can be tested without a database.

The words have precise meanings, and interviews ask for them:

Double	What it is
Dummy	Passed to satisfy a signature, never used
Stub	Returns prepared answers
Spy	A real object that records how it was called
Mock	A stub that also has expectations about the calls
Fake	A working but simplified implementation, such as an in-memory repository

```

@ExtendWith(MockitoExtension.class)
class EmployeeServiceTest {

    @Mock EmployeeRepository repository;
    @Mock EmployeeMapper mapper;

    @InjectMocks EmployeeService service;

    @Test
    void returnsTheEmployeeWhenItExists() {
        Employee employee = new Employee("Ravi", "ravi@example.com", TEN);
        when(repository.findById(1L)).thenReturn(Optional.of(employee));
        when(mapper.toResponse(employee))
            .thenReturn(new EmployeeResponse(1L, "Ravi", "ravi@example.com",
TEN));

        EmployeeResponse response = service.findById(1L);

        assertThat(response.name()).isEqualTo("Ravi");
    }

    @Test
    void throwsWhenTheEmployeeIsMissing() {
        when(repository.findById(99L)).thenReturn(Optional.empty());

        assertThatThrownBy(() -> service.findById(99L))
            .assertInstanceOf(EmployeeNotFoundException.class);
    }
}

```

`@InjectMocks` builds the real service and passes the mocks to its constructor.

Stubbing	Use it for
<code>when(x).thenReturn(v)</code>	A fixed answer
<code>when(x).thenThrow(e)</code>	A failure
<code>when(x).thenAnswer(a)</code>	An answer computed from the arguments
<code>doThrow(e).when(mock).method()</code>	A method returning <code>void</code>

An unused stub fails the test. `MockitoExtension` raises `UnnecessaryStubbingException` when something is stubbed and never called. That is deliberate: a stub nobody uses is either a leftover or a sign the test does not do what its name says.

6. Verifying and capturing

Stubbing decides what a mock returns. Verifying checks how it was called, which matters when the result of the call is the side effect.

```
verify(repository).save(any(Employee.class));
verify(repository, times(2)).findById(1L);
verify(repository, never()).deleteById(anyLong());
verifyNoInteractions(auditClient);
```

`ArgumentCaptor` checks what was passed:

```
ArgumentCaptor<Employee> saved = ArgumentCaptor.forClass(Employee.class);

service.create(new EmployeeRequest("Ravi", "ravi@example.com", TEN));

verify(repository).save(saved.capture());
assertThat(saved.getValue().getEmail()).isEqualTo("ravi@example.com");
```

That is the way to assert on an object the method built internally and never returned.

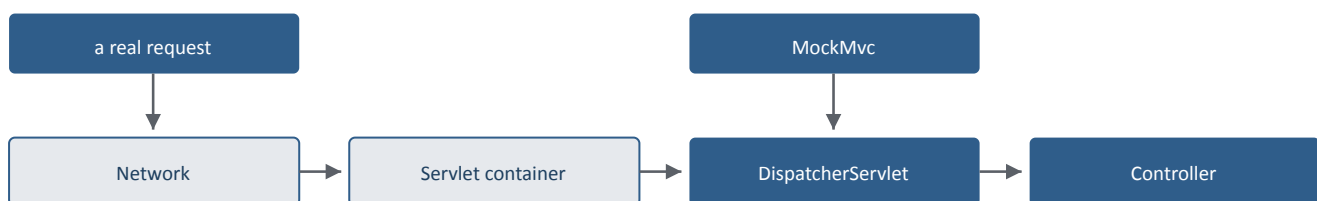
Matchers cannot be mixed with plain values.

```
verify(repository).transfer(eq(1L), 2L);           // InvalidUseOfMatchersException
verify(repository).transfer(eq(1L), eq(2L));       // correct
```

Once one argument uses a matcher, all of them must.

7. MockMvc

MockMvc sends a request through Spring MVC without a server, a port or a socket.



MockMvc never touches these

Skipping the network is what makes it fast, and also what it cannot prove.

There are three ways to get one, and they differ in how much they start:

How	Starts	Use it
<code>standaloneSetup(controller)</code>	Nothing. No Spring context	A controller with no Spring wiring to check
<code>@WebMvcTest</code>	The web layer only	The usual choice

How	Starts	Use it
<code>@SpringBootTest</code> with <code>@AutoConfigureMockMvc</code>	Everything	When the whole context matters

```
@WebMvcTest(EmployeeController.class)
class EmployeeControllerTest {

    @Autowired MockMvc mockMvc;
    @MockitoBean EmployeeService service;

    @Test
    void returnsTheEmployeeAsJson() throws Exception {
        when(service.findById(1L)).thenReturn(
            new EmployeeResponse(1L, "Ravi", "ravi@example.com", TEN));

        mockMvc.perform(get("/api/v1/employees/1"))
            .andExpect(status().isOk())
            .andExpect(content().contentType(MediaType.APPLICATION_JSON))
            .andExpect(jsonPath("$.name").value("Ravi"))
            .andExpect(jsonPath("$.email").value("ravi@example.com"));
    }
}
```

`jsonPath` reaches into the response body:

Expression	Selects
<code>\$.name</code>	A field at the top level
<code>\$.address.city</code>	A nested field
<code>\$.name</code>	A field of the first element
<code>\$.length()</code>	The size of an array
<code>\$.email</code>	Every <code>email</code> anywhere in the document

`andDo(print())` writes the whole exchange to the console, which is the fastest way to find out why an expectation failed.

MockMvc does not prove the service runs. There is no server, so it cannot show that the application starts, that a filter registered outside Spring is applied, or that anything works over a real connection. For those, `@SpringBootTest(webEnvironment = RANDOM_PORT)` starts a real server on a free port.

8. What to mock, and what not to

Mock it	Leave it real
A repository or an HTTP client	Value objects and records
Anything slow, or with side effects	Simple collaborators with no dependencies
A collaborator we own	Anything from the language or a library

Do not mock a type we do not own. A mock of a third-party client is a statement of what we believe that client does. When the belief is wrong the mock still agrees with us, the test passes, and production does not. Wrapping the library in a small interface of our own and mocking that keeps the belief in one place, where an integration test can check it.

Over-mocking turns a test into a copy of the implementation. A test that stubs five collaborators and verifies four calls proves that the code calls what it calls. It fails whenever anything is renamed and catches no defects.

Three signs of it:

```
the setup is longer than the assertion
a rename breaks twenty tests that found no bug
the test reads like the method it tests
```

The usual fix is to use the real object. A value object, a record, an in-memory list or a small pure function costs nothing to construct and makes the test describe behavior rather than wiring.

The pieces together

Piece	What it does
<code>@Test</code> , <code>@BeforeEach</code>	Runs a test from a clean state
<code>@ParameterizedTest</code>	One test, many cases, reported separately
<code>assertThat</code>	An assertion that reads in one direction
<code>@Mock</code> , <code>@InjectMocks</code>	The class under test with stand-ins around it
<code>when(...).thenReturn(...)</code>	What a collaborator answers
<code>verify</code> , <code>ArgumentCaptor</code>	How the collaborator was called, and with what
<code>MockMvc</code> with <code>jsonPath</code>	The HTTP contract, with no server

For example, testing an employee service and its controller takes two classes. The service test uses `@Mock` for the repository and the mapper, stubs `findById` , and checks that a missing employee raises the right exception. The controller test uses `@WebMvcTest` with the service mocked, and checks the status, the content

type and two fields of the JSON. Neither starts a database or a server, and together they run in under a second.

The most useful thing at this point is not to memorize the annotations. Open one existing test, count the lines of setup against the lines of assertion, and see which of the two the test is really about.